

OBJECT ORIENTED PROGRAMMING [CT-260]



NAME: ABDUL RAHEEM SHEIKH

ROLL NO: CT-25192

DEPARTMENT: BCIT

BATCH: 2025

YEAR & SECTION: FSCS-D

LAB: 3

CODES

Q1.

```
#include <iostream>
#include <cstring>
using namespace std;

class Employee {
private:
    char* EmployeeName;
    const int EmployeeId; // Cannot be changed after
    initialization

public:
    // Constructor with member initializer list
    Employee(const char* name, int id) : EmployeeId(id) {
        EmployeeName = new char[strlen(name) + 1];
        strcpy(EmployeeName, name);
    }

    // Destructor
    ~Employee() {
        delete[] EmployeeName;
    }

    // Getter for name
    const char* getEmployeeName() const {
        return EmployeeName;
    }

    // Getter for ID
```

```
int getEmployeeId() const {
    return EmployeeId;
}

// Setter for name only
void setEmployeeName(const char* name) {
    delete[] EmployeeName;
    EmployeeName = new char[strlen(name) + 1];
    strcpy(EmployeeName, name);
}
};

int main() {
    Employee Employee1("Ali", 101);
    Employee Employee2("Sara", 102);
    Employee Employee3("Jhon", 103);

    Employee1.setEmployeeName("Ahmed");

    cout << "Employee 1: " <<
Employee1.getEmployeeName()
    << ", ID: " << Employee1.getEmployeeId() << endl;

    cout << "Employee 2: " <<
Employee2.getEmployeeName()
    << ", ID: " << Employee2.getEmployeeId() << endl;

    cout << "Employee 3: " <<
Employee3.getEmployeeName()
    << ", ID: " << Employee3.getEmployeeId() << endl;

    return 0;
}
```

Q2.

```
#include <iostream>
using namespace std;

class DynamicArray {
private:
    int* arr;
    int capacity;
    int currentSize;

public:
    DynamicArray(int size) {
        capacity = size;
        currentSize = 0;
        arr = new int[capacity];

        for(int i = 0; i < capacity; i++)
            arr[i] = 0;
    }

    ~DynamicArray() {
        delete[] arr;
    }

    void push(int value) {
        if(currentSize < capacity) {
            arr[currentSize++] = value;
        } else {
            cout << "Array is full!" << endl;
        }
    }
}
```

```
    }

    int size() const {
        return currentSize;
    }

    void display() const {
        for(int i = 0; i < currentSize; i++)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    DynamicArray obj(5);
    obj.push(10);
    obj.push(20);
    obj.push(30);

    obj.display();
    cout << "Size: " << obj.size() << endl;

    return 0;
}
```

Q3.

```
#include <iostream>
using namespace std;

class Account {
private:
    int account_no;
    double account_bal;
    int security_code;

    static int count;

public:
    void initialize(int no, double bal, int code) {
        account_no = no;
        account_bal = bal;
        security_code = code;
        count++;
    }

    void display() const {
        cout << "Account No: " << account_no << endl;
        cout << "Balance: " << account_bal << endl;
        cout << "Security Code: " << security_code << endl;
    }

    static int getCount() {
        return count;
    }
};
```

```
int Account::count = 0;

int main() {
    Account a1, a2;

    a1.initialize(1001, 5000.75, 1234);
    a2.initialize(1002, 7000.50, 5678);

    a1.display();
    a2.display();

    cout << "Total Accounts Created: " <<
    Account::getCount() << endl;

    return 0;
}
```

Q4.

```
#include <iostream>
using namespace std;

class Student {
private:
    string name;
    const int rollNumber; // Cannot change
    float gpa;

public:
```

```
Student(string n, int r, float g) : rollNumber(r) {  
    name = n;  
    gpa = g;  
}  
  
void setGPA(float g) {  
    gpa = g;  
}  
  
float getGPA() const { // const member function  
    return gpa;  
}  
  
int getRollNumber() const {  
    return rollNumber;  
}  
  
void display() const {  
    cout << "Name: " << name << endl;  
    cout << "Roll Number: " << rollNumber << endl;  
    cout << "GPA: " << gpa << endl;  
}  
};  
  
int main() {  
    Student s1("Ali", 101, 3.5);  
    s1.setGPA(3.8);  
    s1.display();  
    return 0;  
}
```


Q5.

```
#include <iostream>
using namespace std;

class HotelMercato {
private:
    string customerName;
    int days;
    static const double rentPerDay; // Fixed rent decided by
committee
    double totalRent;

    void calculateRent() {
        if(days > 7)
            totalRent = (days - 1) * rentPerDay;
        else
            totalRent = days * rentPerDay;
    }

public:
    HotelMercato(string name, int d) {
        customerName = name;
        days = d;
        calculateRent();
    }

    void display() const { // Cannot modify data
        cout << "Customer Name: " << customerName << endl;
        cout << "Days: " << days << endl;
        cout << "Rent: " << totalRent << endl;
    }
};
```

```
    }  
};
```

```
const double HotelMercato::rentPerDay = 1000.85;
```

```
int main() {  
    HotelMercato c1("Ahmed", 5);  
    HotelMercato c2("Sara", 10);  
  
    c1.display();  
    cout << endl;  
    c2.display();  
  
    return 0;  
}
```

OUTPUTS

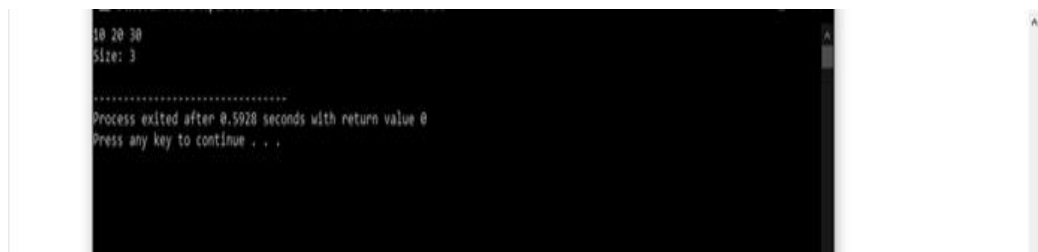
Q1.



```
Employee 1: Ahmed, ID: 101
Employee 2: Sara, ID: 102
Employee 3: John, ID: 103

.....
Process exited after 0.553 seconds with return value 0
Press any key to continue . . .
```

Q2.



```
10 20 30
Size: 3

.....
Process exited after 0.5928 seconds with return value 0
Press any key to continue . . .
```

Q.3

```
100 Account No: 1001
    Balance: 5000.75
    Security Code: 1234
    Account No: 1002
    Balance: 7000.5
    Security Code: 5678
    Total Accounts Created: 2

.....
Process exited after 0.3977 seconds with return value 0
Press any key to continue . . .
```

Q4.

```
Name: Ali
Roll Number: 101
GPA: 3.8

.....
Process exited after 0.1801 seconds with return value 0
Press any key to continue . . .
```

Q5.

```
Customer Name: Ahmad
Days: 5
Rent: 5004.25
.....
Customer Name: Sara
Days: 10
Rent: 9007.05

.....
Process exited after 0.7369 seconds with return value 0
Press any key to continue . . .
```

